

Instrukcja implementacji wzorca projektowego

Pełnomocnik / Proxy

1 Nazwa wzorca

Pełnomocnik, znany również pod angielską nazwą **Proxy**, jest strukturalnym wzorcem projektowym. Jego zadaniem jest utworzenie obiektu zastępczego, który reprezentuje inny obiekt i kontroluje dostęp do niego.

Zamiast korzystać bezpośrednio z właściwego obiektu, program korzysta z jego pełnomocnika. Pełnomocnik powinien udostępniać taki sam sposób użycia jak obiekt rzeczywisty. Dzięki temu z perspektywy reszty programu nie trzeba zmieniać sposobu wywoływania metod.

Najprościej można powiedzieć, że **pełnomocnik stoi pomiędzy użytkownikiem programu a obiektem rzeczywistym**. Może na przykład zdecydować, kiedy utworzyć właściwy obiekt i kiedy przekazać mu wykonanie zadania.

W tej instrukcji wzorec zostanie opisany na prostym przykładzie dokumentu. Dokument właściwy będzie tworzony dopiero wtedy, gdy użytkownik pierwszy raz będzie chciał go wyświetlić.

2 Idea działania wzorca

Wzorec Pełnomocnik składa się z trzech podstawowych elementów:

1. wspólnej klasy bazowej,
2. obiektu rzeczywistego,
3. obiektu pełnomocnika.

2.1 Wspólna klasa bazowa

Wspólna klasa bazowa określa, jaką metodę powinien posiadać zarówno obiekt rzeczywisty, jak i pełnomocnik.

W przykładzie z dokumentem będzie to metoda służąca do wyświetlenia dokumentu. Można ją nazwać `display()`.

Dzięki wspólnej klasie bazowej zarówno prawdziwy dokument, jak i jego pełnomocnik mają tę samą metodę. To jest bardzo ważne, ponieważ użytkownik programu może wywołać `display()` na pełnomocniku tak samo, jak wywołałby ją na prawdziwym dokumencie.

W Pythonie wspólną klasę bazową można przygotować za pomocą modułu `abc`. Moduł ten pozwala tworzyć klasy abstrakcyjne, czyli takie, które opisują wymagane metody, ale same nie wykonują jeszcze właściwej pracy.

2.2 Obiekt rzeczywisty

Obiekt rzeczywisty to klasa, która wykonuje właściwe zadanie.

W naszym przykładzie będzie to klasa reprezentująca prawdziwy dokument. Jej zadaniem będzie:

- zapamiętanie nazwy dokumentu,
- zasymulowanie załadowania dokumentu,
- wyświetlenie dokumentu po wywołaniu metody `display()`.

Nie trzeba wczytywać prawdziwego pliku z dysku. Wystarczą komunikaty wypisywane za pomocą `print()`, ponieważ celem zadania jest pokazanie działania wzorca, a nie obsługa plików.

2.3 Pełnomocnik

Pełnomocnik to klasa, która reprezentuje prawdziwy dokument, ale nie tworzy go od razu.

Pełnomocnik powinien:

- zapamiętać nazwę dokumentu,
- posiadać pole przechowujące prawdziwy dokument,
- na początku ustawić to pole na `None`,
- utworzyć prawdziwy dokument dopiero przy pierwszym wywołaniu metody `display()`,
- przy kolejnych wywołaniach korzystać już z wcześniej utworzonego dokumentu.

Taki sposób działania nazywa się **leniwą inicjalizacją**. Oznacza to, że obiekt jest tworzony dopiero wtedy, gdy naprawdę jest potrzebny.

3 Instrukcja implementacji w Pythonie

3.1 Krok 1: Przygotowanie pliku

Utwórz jeden plik Pythona, na przykład:

`proxy_document.py`

Całą implementację można wykonać w tym jednym pliku. Nie trzeba tworzyć kilku modułów ani osobnych folderów.

3.2 Krok 2: Zaimportowanie potrzebnych elementów

Na początku pliku należy zaimportować elementy potrzebne do stworzenia klasy abstrakcyjnej.

Potrzebny będzie moduł `abc`, a dokładniej:

- `ABC` – do tworzenia klasy abstrakcyjnej,
- `abstractmethod` – do oznaczenia metody, którą trzeba zaimplementować w klasach potomnych.

3.3 Krok 3: Utworzenie wspólnej klasy bazowej

Następnie utwórz klasę bazową o nazwie `Document`.

Klasa ta powinna dziedziczyć po `ABC`. W środku klasy należy zdefiniować metodę `display()` i oznaczyć ją jako abstrakcyjną za pomocą `@abstractmethod`.

Metoda `display()` nie musi wykonywać żadnej konkretnej operacji w klasie bazowej. Jej zadaniem jest tylko wymuszenie, aby każda klasa dokumentu posiadała taką metodę.

Na tym etapie należy zapamiętać:

- `Document` to wspólna podstawa,
- `display()` to metoda, którą muszą mieć obie klasy,
- klasa bazowa nie wyświetla jeszcze dokumentu.

3.4 Krok 4: Utworzenie klasy rzeczywistego dokumentu

Utwórz klasę `RealDocument`. Powinna ona dziedziczyć po klasie `Document`.

W konstruktorze klasy `RealDocument` należy przyjąć nazwę pliku, na przykład `filename`. Następnie należy zapisać tę nazwę w polu obiektu, na przykład `self.filename`.

W konstruktorze warto też wypisać komunikat informujący, że dokument jest ładowany. Dzięki temu podczas uruchomienia programu będzie widać, kiedy prawdziwy dokument faktycznie zostaje utworzony.

Klasa `RealDocument` powinna mieć także metodę `display()`. Ta metoda powinna wypisać komunikat, że dokument o podanej nazwie jest wyświetlany.

Przykładowa logika tej klasy jest następująca:

1. podczas tworzenia obiektu zapamiętaj nazwę dokumentu,
2. wypisz komunikat o ładowaniu dokumentu,
3. w metodzie `display()` wypisz komunikat o wyświetlaniu dokumentu.

3.5 Krok 5: Utworzenie klasy pełnomocnika

Utwórz klasę `DocumentProxy`. Ona również powinna dziedziczyć po klasie `Document`.

W konstruktorze klasy `DocumentProxy` należy:

1. przyjąć nazwę pliku,
2. zapisać ją w polu, na przykład `self.filename`,
3. utworzyć pole na prawdziwy dokument, na przykład `self.real_document`,
4. ustawić to pole na `None`.

Ustawienie wartości `None` oznacza, że prawdziwy dokument jeszcze nie istnieje.

Następnie w klasie `DocumentProxy` należy zaimplementować metodę `display()`.

Metoda `display()` w pełnomocniku powinna działać tak:

1. sprawdź, czy `self.real_document` ma wartość `None`,

2. jeżeli tak, utwórz obiekt klasy `RealDocument` i zapisz go w `self.real_document`,
3. jeżeli nie, nie twórz nowego dokumentu,
4. wywołaj metodę `display()` na obiekcie zapisanym w `self.real_document`.

Najważniejszy fragment logiki jest taki: prawdziwy dokument ma zostać utworzony tylko raz, przy pierwszym użyciu. Przy drugim i kolejnym wywołaniu metody `display()` pełnomocnik powinien użyć już istniejącego obiektu.

3.6 Krok 6: Utworzenie prostego programu testowego

Na końcu pliku utwórz krótki program testowy.

Program powinien wykonać następujące kroki:

1. utworzyć pełnomocnika dokumentu, na przykład dla pliku `raport.pdf`,
2. wypisać komunikat, że pełnomocnik został utworzony,
3. wywołać metodę `display()` pierwszy raz,
4. wywołać metodę `display()` drugi raz.

Ważne: w programie testowym należy tworzyć obiekt klasy `DocumentProxy`, a nie `RealDocument`. Dzięki temu będzie widać działanie wzorca Pełnomocnik.

4 Przykład do zaimplementowania

Zaimplementuj prosty system wyświetlania dokumentów.

Program powinien zawierać trzy klasy:

1. `Document` – wspólna klasa bazowa,
2. `RealDocument` – prawdziwy dokument,
3. `DocumentProxy` – pełnomocnik dokumentu.

4.1 Wymagania dla klasy `Document`

Klasa `Document` powinna:

- być klasą abstrakcyjną,
- zawierać abstrakcyjną metodę `display()`,
- nie wykonywać właściwego wyświetlania dokumentu.

4.2 Wymagania dla klasy RealDocument

Klasa RealDocument powinna:

- dziedziczyć po klasie Document,
- mieć konstruktor przyjmujący nazwę pliku,
- zapamiętywać nazwę pliku,
- wypisywać komunikat o ładowaniu dokumentu w momencie tworzenia obiektu,
- posiadać metodę `display()`,
- w metodzie `display()` wypisywać komunikat o wyświetlaniu dokumentu.

4.3 Wymagania dla klasy DocumentProxy

Klasa DocumentProxy powinna:

- dziedziczyć po klasie Document,
- mieć konstruktor przyjmujący nazwę pliku,
- zapamiętywać nazwę pliku,
- posiadać pole na prawdziwy dokument,
- ustawić to pole początkowo na `None`,
- posiadać metodę `display()`,
- przy pierwszym wywołaniu `display()` utworzyć obiekt RealDocument,
- przy kolejnych wywołaniach `display()` nie tworzyć nowego obiektu,
- przekazywać wywołanie `display()` do prawdziwego dokumentu.

4.4 Oczekiwany przebieg działania programu

Po uruchomieniu programu powinno być widać następującą kolejność zdarzeń:

1. tworzony jest pełnomocnik dokumentu,
2. prawdziwy dokument nie jest jeszcze ładowany,
3. przy pierwszym wywołaniu `display()` dokument zostaje załadowany,
4. dokument zostaje wyświetlony,
5. przy drugim wywołaniu `display()` dokument nie jest ładowany ponownie,
6. dokument zostaje wyświetlony drugi raz.

Przykładowy sens komunikatów w konsoli może być taki:

- Utworzono pełnomocnika dla dokumentu `raport.pdf`
- Pierwsze wyświetlenie dokumentu
- Ładowanie dokumentu `raport.pdf`
- Wyświetlanie dokumentu `raport.pdf`
- Drugie wyświetlenie dokumentu
- Wyświetlanie dokumentu `raport.pdf`

Zwróć uwagę, że komunikat o ładowaniu dokumentu powinien pojawić się tylko raz. To pokazuje, że pełnomocnik poprawnie utworzył prawdziwy obiekt dopiero przy pierwszym użyciu i zapamiętał go na później.

Najważniejsze jest to, aby pełnomocnik nie wykonywał całej pracy samodzielnie. Pełnomocnik ma tylko kontrolować dostęp do prawdziwego dokumentu. Właściwe wyświetlanie powinno być wykonywane przez obiekt `RealDocument`.